

Java Memory Management – Proof of Concept (POC)

Objective

This Proof of Concept demonstrates how **Java allocates and manages memory** during program execution. It explains memory areas, object lifecycle, and important JVM behaviors using a simple Java program.

Java Example Code

```
public class JavaMemoryPOC {

    static int staticVar = 100;
    int instanceVar = 50;

    public static void main(String[] args) {

        // Stack Memory (Primitive)
        int primitive = 10;

        // Heap Memory (Object)
        JavaMemoryPOC obj = new JavaMemoryPOC();
        obj.show();

        // String Constant Pool
        String s1 = "Java";
        String s2 = "Java";
        String s3 = new String("Java");

        System.out.println(s1 == s2);
        System.out.println(s1 == s3);

        // Pass by Value
        int x = 20;
        changePrimitive(x);
        System.out.println(x);

        JavaMemoryPOC obj2 = new JavaMemoryPOC();
        changeObject(obj2);
        System.out.println(obj2.instanceVar);
    }
}
```

```
// Garbage Collection Eligibility
JavaMemoryPOC temp = new JavaMemoryPOC();
temp = null;
System.gc();
}

void show() {
    int localVar = 30;
    System.out.println(localVar);
    System.out.println(instanceVar);
    System.out.println(staticVar);
}

static void changePrimitive(int val) {
    val = 100;
}

static void changeObject(JavaMemoryPOC ref) {
    ref.instanceVar = 999;
    ref = null;
}
}
```

Memory Areas Explained

Stack Memory

- Stores **local variables**, **method calls**, and **object references**
- Memory is freed automatically when method execution ends

Examples:

```
int primitive = 10;
JavaMemoryPOC obj;
```

Heap Memory

- Stores **objects** and **instance variables**
- Managed by Garbage Collector

Examples:

```
new JavaMemoryPOC();  
instanceVar
```

Method Area / Metaspace

- Stores **class metadata**, **methods**, and **static variables**

Example:

```
static int staticVar = 100;
```

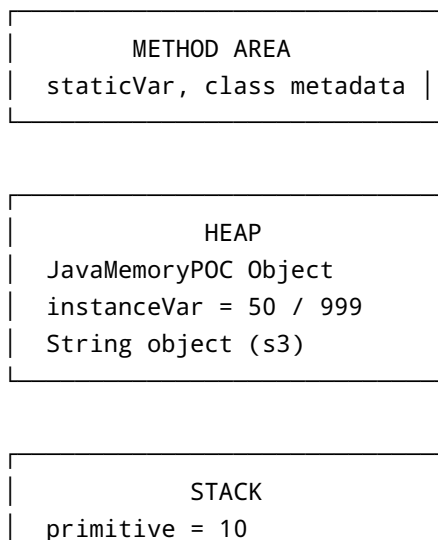
String Constant Pool (SCP)

- Stores string literals for reuse

Examples:

```
String s1 = "Java";  
String s2 = "Java";      // Same SCP reference  
String s3 = new String("Java"); // Heap object
```

JVM Memory Diagram



```
x = 20  
obj, obj2 references  
localVar = 30
```

```
STRING CONSTANT POOL  
"Java" → s1, s2
```



Object Lifecycle

1. Object created using `new`
2. Reference stored in Stack
3. Object lives in Heap
4. Reference removed (`null`)
5. Object becomes eligible for Garbage Collection



Pass By Value Explanation

- Java always passes **copies of values**
- For objects, the **reference copy** is passed

```
changePrimitive(x); // No change  
changeObject(obj2); // Object modified
```



Program Output

```
30  
50  
100  
true  
false  
20  
999
```

Key Takeaways

- Stack is fast and short-lived
- Heap stores objects and survives method calls
- Strings are optimized using SCP
- Java uses pass-by-value
- Garbage Collection frees unused heap memory

 **This POC is suitable for interviews, training, and Java fundamentals demonstrations.**